Java Interview Preparation

Part 4: Top 100 Basic Core & Number Logic Questions

Section 1: Basic Number Manipulation & Logic (Q1–Q25)

Q1: Check if a number is Even or Odd using Bitwise operator (&).

```
// Time: O(1), Space: O(1)
public static boolean isEven(int n) {
    return (n & 1) == 0;
}
```

Q2: Check if a given number is positive, negative, or zero.

```
public static String checkSign(int n) {
    if (n > 0) return "Positive";
    if (n < 0) return "Negative";
    return "Zero";
}
```

Q3: Swap two numbers without using a third (temp) variable (Arithmetic & Bitwise).

```
// Method 1: Arithmetic
public static void swapNumbers(int a, int b) {
    a = a + b;
    b = a - b;
    a = a - b;
}

// Method 2: XOR Bitwise
public static void swapBitwise(int a, int b) {
    a = a ^ b;
    b = a ^ b;
    a = a ^ b;
}
```

Q4: Find the largest among three numbers using ternary operators.

```
public static int findMax(int a, int b, int c) {
    return (a >= b && a >= c) ? a : (b >= c ? b : c);
}
```

Q5: Check if a given year is a Leap Year.

```
public static boolean isLeapYear(int year) {
    return (year % 400 == 0) || (year % 4 == 0 && year % 100 != 0);
}
```

Q6: Check if a number is a Palindrome (e.g., 12321).

```
// Time: O(log10 N), Space: O(1)
public static boolean isPalindromeNumber(int n) {
    if (n < 0 || (n % 10 == 0 && n != 0)) return false;
    int reversed = 0, original = n;
    while (n > 0) {
        reversed = reversed * 10 + n % 10;
        n /= 10;
    }
    return original == reversed;
}
```

Q7: Reverse an integer and check for 32-bit integer overflow.

```
// Time: O(log10 N), Space: O(1)
public static int reverseInteger(int x) {
    int rev = 0;
    while (x != 0) {
        int pop = x % 10;
        x /= 10;
        if (rev > Integer.MAX_VALUE / 10 || (rev == Integer.MAX_VALUE / 10 && pop > 7)) return 0
        if (rev < Integer.MIN_VALUE / 10 || (rev == Integer.MIN_VALUE / 10 && pop < -8)) return
        rev = rev * 10 + pop;
    }
    return rev;
}
```

Q8: Check if a number is an Armstrong Number (e.g., 153 = 1^3 + 5^3 + 3^3).

```
public static boolean isArmstrong(int n) {
    int original = n, sum = 0;
    int digits = String.valueOf(n).length();
    while (n > 0) {
        int digit = n % 10;
        sum += Math.pow(digit, digits);
        n /= 10;
    }
    return original == sum;
}
```

Q9: Print Fibonacci series up to N terms.

```
public static void printFibonacci(int n) {
    int a = 0, b = 1;
    for (int i = 0; i < n; i++) {
        System.out.print(a + " ");
        int next = a + b;
        a = b;
        b = next;
    }
}
```

Q10: Find Nth Fibonacci number using Recursion with Memoization.

```
public static long fibonacciMemo(int n, long[] memo) {
    if (n <= 1) return n;
    if (memo[n] != 0) return memo[n];
    return memo[n] = fibonacciMemo(n - 1, memo) + fibonacciMemo(n - 2, memo);
}
```

Q11: Compute Factorial of a number iteratively and recursively.

```
// Iterative
public static long factorialIterative(int n) {
    long fact = 1;
    for (int i = 2; i <= n; i++) fact *= i;
    return fact;
}

// Recursive
public static long factorialRecursive(int n) {
    if (n <= 1) return 1;
    return n * factorialRecursive(n - 1);
}
```

Q12: Check if a number is Prime.

```
// Time: O(sqrt(N)), Space: O(1)
public static boolean isPrime(int n) {
    if (n <= 1) return false;
    if (n <= 3) return true;
    if (n % 2 == 0 || n % 3 == 0) return false;
    for (int i = 5; i * i <= n; i += 6) {
        if (n % i == 0 || n % (i + 2) == 0) return false;
    }
    return true;
}
```

Q13: Print all Prime Numbers in a range [1, N] using Sieve of Eratosthenes.

```
// Time: O(N log(log N)), Space: O(N)
public static void sieveOfEratosthenes(int n) {
    boolean[] prime = new boolean[n + 1];
    Arrays.fill(prime, true);
    for (int p = 2; p * p <= n; p++) {
        if (prime[p]) {
            for (int i = p * p; i <= n; i += p) prime[i] = false;
        }
    }
    for (int i = 2; i <= n; i++) {
        if (prime[i]) System.out.print(i + " ");
    }
}
```

Q14: Find Greatest Common Divisor (GCD/HCF) using Euclidean Algorithm.

```
public static int gcd(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}
```

Q15: Find Least Common Multiple (LCM) of two numbers.

```
public static int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}
```

Q16: Count total number of digits in an integer.

```
public static int countDigits(int n) {
    if (n == 0) return 1;
    return (int) Math.log10(Math.abs(n)) + 1;
}
```

Q17: Calculate the sum of digits of a number.

```
public static int sumOfDigits(int n) {
    int sum = 0;
    while (n != 0) {
        sum += Math.abs(n % 10);
        n /= 10;
    }
    return sum;
}
```

Q18: Check if a number is a Perfect Number (Sum of proper divisors equals the number).

```
public static boolean isPerfectNumber(int n) {
    if (n <= 1) return false;
    int sum = 1;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            sum += i;
            if (i * i != n) sum += n / i;
        }
    }
    return sum == n;
}
```

Q19: Check if a number is a Strong Number (Sum of factorials of digits equals the number).

```
public static boolean isStrongNumber(int n) {
    int original = n, sum = 0;
    while (n > 0) {
        sum += factorialIterative(n % 10);
        n /= 10;
    }
    return original == sum;
}
```

Q20: Check if a number is an Automorphic Number (e.g., 25^2 = 625, ends in 25).

```
public static boolean isAutomorphic(int n) {
    long square = (long) n * n;
    while (n > 0) {
        if (n % 10 != square % 10) return false;
        n /= 10;
        square /= 10;
    }
    return true;
}
```

Q21: Check if a number is a Harshad / Niven Number (Divisible by sum of its digits).

```
public static boolean isHarshad(int n) {
    int sum = sumOfDigits(n);
    return n % sum == 0;
}
```

Q22: Check if a number is a Neon Number (Sum of digits of square equals the number, e.g., 9^2 = 81 -> 8 + 1 = 9).

```
public static boolean isNeonNumber(int n) {
    int square = n * n;
    return sumOfDigits(square) == n;
}
```

Q23: Check if a number is a Spy Number (Sum of digits equals product of digits).

```
public static boolean isSpyNumber(int n) {
    int sum = 0, product = 1;
    while (n > 0) {
        int d = n % 10;
        sum += d;
        product *= d;
        n /= 10;
    }
    return sum == product;
}
```

Q24: Check if a number is a Happy Number.

```
public static boolean isHappy(int n) {
    Set<Integer> seen = new HashSet<>();
    while (n != 1 && !seen.contains(n)) {
        seen.add(n);
        int sum = 0;
        while (n > 0) {
            int d = n % 10;
            sum += d * d;
            n /= 10;
        }
        n = sum;
    }
    return n == 1;
}
```

Q25: Find Prime Factors of a number.

```
public static void printPrimeFactors(int n) {
    while (n % 2 == 0) {
        System.out.print(2 + " ");
        n /= 2;
    }
    for (int i = 3; i * i <= n; i += 2) {
        while (n % i == 0) {
            System.out.print(i + " ");
            n /= i;
        }
    }
    if (n > 2) System.out.print(n);
}
```

Section 2: Numbers, Sequences & Conversions (Q26–Q50)

Q26: Convert Decimal to Binary string.

```
public static String decimalToBinary(int n) {
    StringBuilder sb = new StringBuilder();
    if (n == 0) return "0";
    while (n > 0) {
        sb.append(n % 2);
        n /= 2;
    }
    return sb.reverse().toString();
}
```

Q27: Convert Binary string to Decimal integer.

```
public static int binaryToDecimal(String binary) {
    int decimal = 0, power = 0;
    for (int i = binary.length() - 1; i >= 0; i--) {
        if (binary.charAt(i) == '1') {
            decimal += Math.pow(2, power);
        }
        power++;
    }
    return decimal;
}
```

Q28: Convert Decimal to Hexadecimal without using built-in methods.

```
public static String decimalToHex(int n) {
    if (n == 0) return "0";
    char[] hexChars = "0123456789ABCDEF".toCharArray();
    StringBuilder sb = new StringBuilder();
    while (n > 0) {
        sb.append(hexChars[n % 16]);
        n /= 16;
    }
    return sb.reverse().toString();
}
```

Q29: Convert Octal to Decimal.

```
public static int octalToDecimal(int octal) {
    int decimal = 0, power = 0;
    while (octal > 0) {
        decimal += (octal % 10) * Math.pow(8, power++);
        octal /= 10;
    }
    return decimal;
}
```

Q30: Count set bits (number of 1s) in an integer (Brian Kernighan's Algorithm).

```
public static int countSetBits(int n) {
    int count = 0;
    while (n > 0) {
        n &= (n - 1); // Clears the lowest set bit
        count++;
    }
    return count;
}
```

Q31: Check if a power of two.

```
public static boolean isPowerOfTwo(int n) {
    return n > 0 && (n & (n - 1)) == 0;
}
```

Q32: Add two binary strings and return sum as a binary string.

```
public static String addBinary(String a, String b) {
    StringBuilder sb = new StringBuilder();
    int i = a.length() - 1, j = b.length() - 1, carry = 0;
    while (i >= 0 || j >= 0 || carry != 0) {
        int sum = carry;
        if (i >= 0) sum += a.charAt(i--) - '0';
        if (j >= 0) sum += b.charAt(j--) - '0';
        sb.append(sum % 2);
        carry = sum / 2;
    }
    return sb.reverse().toString();
}
```

Q33: Calculate A^B (Power of a number) using Binary Exponentiation.

```
// Time: O(log B), Space: O(1)
public static double power(double a, int b) {
    long N = b;
    if (N < 0) {
        a = 1 / a;
        N = -N;
    }
    double ans = 1;
    double currentProduct = a;
    for (long i = N; i > 0; i /= 2) {
        if (i % 2 == 1) ans *= currentProduct;
        currentProduct *= currentProduct;
    }
    return ans;
}
```

Q34: Check if a number is a Perfect Square without using Math.sqrt().

```
public static boolean isPerfectSquare(int num) {
    long left = 1, right = num;
    while (left <= right) {
        long mid = left + (right - left) / 2;
        if (mid * mid == num) return true;
        if (mid * mid < num) left = mid + 1;
        else right = mid - 1;
    }
    return false;
}
```

Q35: Check if a number is a Sunny Number (N + 1 is a perfect square).

```
public static boolean isSunnyNumber(int n) {
    return isPerfectSquare(n + 1);
}
```

Q36: Check if a number is a Trimorphic Number (Cube of the number ends in the number itself).

```
public static boolean isTrimorphic(int n) {
    long cube = (long) n * n * n;
    while (n > 0) {
        if (n % 10 != cube % 10) return false;
        n /= 10;
        cube /= 10;
    }
    return true;
}
```

Q37: Print all Armstrong numbers between 1 and 1000.

```
public static void printArmstrongInRange(int start, int end) {
    for (int i = start; i <= end; i++) {
        if (isArmstrong(i)) System.out.print(i + " ");
    }
}
```

Q38: Find sum of all prime numbers between 1 and N.

```
public static long sumOfPrimes(int n) {
    long sum = 0;
    for (int i = 2; i <= n; i++) {
        if (isPrime(i)) sum += i;
    }
    return sum;
}
```

Q39: Find trailing zeros in N! (Factorial).

```
// Time: O(log5 N), Space: O(1)
public static int trailingZerosInFactorial(int n) {
    int count = 0;
    for (int i = 5; n / i >= 1; i *= 5) {
        count += n / i;
    }
    return count;
}
```

Q40: Find Nth Catalan Number.

```
public static long catalanNumber(int n) {
    long[] c = new long[n + 1];
    c[0] = 1; c[1] = 1;
    for (int i = 2; i <= n; i++) {
        c[i] = 0;
        for (int j = 0; j < i; j++) {
            c[i] += c[j] * c[i - 1 - j];
        }
    }
    return c[n];
}
```

Q41: Check if a number is a Ugly Number (Prime factors limited to 2, 3, 5).

```
public static boolean isUgly(int n) {
    if (n <= 0) return false;
    int[] factors = {2, 3, 5};
    for (int f : factors) {
        while (n % f == 0) n /= f;
    }
    return n == 1;
}
```

Q42: Check if two numbers are Friendly Pairs / Amicable Numbers (Sum of proper divisors ratio is equal).

```
public static boolean areAmicable(int a, int b) {
    return sumProperDivisors(a) == b && sumProperDivisors(b) == a;
}

private static int sumProperDivisors(int n) {
    int sum = 1;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            sum += i;
            if (i * i != n) sum += n / i;
        }
    }
    return sum;
}
```

Q43: Check if a number is an Abundant Number (Sum of proper divisors > Number).

```
public static boolean isAbundant(int n) {
    return sumProperDivisors(n) > n;
}
```

Q44: Check if a number is a Deficient Number (Sum of proper divisors < Number).

```
public static boolean isDeficient(int n) {
    return sumProperDivisors(n) < n;
}
```

Q45: Check if a number is a Kaprekar Number (e.g., 45^2 = 2025 -> 20 + 25 = 45).

```
public static boolean isKaprekar(int n) {
    if (n == 1) return true;
    long sq = (long) n * n;
    String sqStr = String.valueOf(sq);
    int len = sqStr.length();
    for (int i = 1; i < len; i++) {
        long parts1 = Long.parseLong(sqStr.substring(0, i));
        long parts2 = Long.parseLong(sqStr.substring(i));
        if (parts2 != 0 && parts1 + parts2 == n) return true;
    }
    return false;
}
```

Q46: Multiply two integers without using the * operator.

```
public static int multiplyWithoutStar(int a, int b) {
    int result = 0;
    int absB = Math.abs(b);
    for (int i = 0; i < absB; i++) result += Math.abs(a);
    return (a < 0 ^ b < 0) ? -result : result;
}
```

Q47: Divide two integers without using /, *, or % operators.

```
public static int divide(int dividend, int divisor) {
    if (dividend == Integer.MIN_VALUE && divisor == -1) return Integer.MAX_VALUE;
    boolean negative = (dividend < 0) ^ (divisor < 0);
    long dvd = Math.abs((long) dividend);
    long dvs = Math.abs((long) divisor);
    int result = 0;
    while (dvd >= dvs) {
        long temp = dvs, multiple = 1;
        while (dvd >= (temp << 1)) {
            temp <<= 1;
            multiple <<= 1;
        }
        dvd -= temp;
        result += multiple;
    }
    return negative ? -result : result;
}
```

Q48: Find the square root of a number using Newton-Raphson Method.

```
public static double sqrtNewton(double n) {
    double x = n;
    double root;
    while (true) {
        root = 0.5 * (x + (n / x));
        if (Math.abs(root - x) < 1e-7) break;
        x = root;
    }
    return root;
}
```

Q49: Check if a number is a Pronic Number (Product of two consecutive integers N = K(K+1)).

```
public static boolean isPronic(int n) {
    for (int i = 0; i * (i + 1) <= n; i++) {
        if (i * (i + 1) == n) return true;
    }
    return false;
}
```

Q50: Check if a number is a Disarium Number (Sum of digits powered by their positions equals the number, e.g., 175 = 1^1 + 7^2 + 5^3).

```
public static boolean isDisarium(int n) {
    String str = String.valueOf(n);
    int sum = 0;
    for (int i = 0; i < str.length(); i++) {
        int digit = str.charAt(i) - '0';
        sum += Math.pow(digit, i + 1);
    }
    return sum == n;
}
```

Section 3: Strings, Characters & Basic Arrays (Q51–Q75)

Q51: Reverse a String without built-in reverse().

```
public static String reverseString(String s) {
    char[] arr = s.toCharArray();
    int l = 0, r = arr.length - 1;
    while (l < r) {
        char temp = arr[l];
        arr[l++] = arr[r];
        arr[r--] = temp;
    }
    return new String(arr);
}
```

Q52: Check if a String is a Palindrome.

```
public static boolean isStringPalindrome(String s) {
    int l = 0, r = s.length() - 1;
    while (l < r) {
        if (s.charAt(l++) != s.charAt(r--)) return false;
    }
    return true;
}
```

Q53: Count Vowels and Consonants in a string.

```
public static void countVowelsAndConsonants(String s) {
    int vowels = 0, consonants = 0;
    s = s.toLowerCase();
    for (char c : s.toCharArray()) {
        if (c >= 'a' && c <= 'z') {
            if ("aeiou".indexOf(c) != -1) vowels++;
            else consonants++;
        }
    }
    System.out.println("Vowels: " + vowels + ", Consonants: " + consonants);
}
```

Q54: Remove all spaces from a string.

```
public static String removeSpaces(String s) {
    StringBuilder sb = new StringBuilder();
    for (char c : s.toCharArray()) {
        if (!Character.isWhitespace(c)) sb.append(c);
    }
    return sb.toString();
}
```

Q55: Remove all special characters/symbols from a string.

```
public static String removeSpecialChars(String s) {
    return s.replaceAll("[^a-zA-Z0-9]", "");
}
```

Q56: Capitalize the first letter of each word in a string.

```
public static String capitalizeWords(String s) {
    String[] words = s.split("\\s+");
    StringBuilder sb = new StringBuilder();
    for (String w : words) {
        if (!w.isEmpty()) {
            sb.append(Character.toUpperCase(w.charAt(0)))
              .append(w.substring(1).toLowerCase())
              .append(" ");
        }
    }
    return sb.toString().trim();
}
```

Q57: Toggle case of each character in a string.

```
public static String toggleCase(String s) {
    char[] arr = s.toCharArray();
    for (int i = 0; i < arr.length; i++) {
        if (Character.isUpperCase(arr[i])) arr[i] = Character.toLowerCase(arr[i]);
        else if (Character.isLowerCase(arr[i])) arr[i] = Character.toUpperCase(arr[i]);
    }
    return new String(arr);
}
```

Q58: Find occurrence frequency of each character in a string.

```
public static void charFrequency(String s) {
    int[] freq = new int[256];
    for (char c : s.toCharArray()) freq[c]++;
    for (int i = 0; i < 256; i++) {
        if (freq[i] > 0) System.out.println((char) i + " : " + freq[i]);
    }
}
```

Q59: Find non-repeating characters in a string.

```
public static void printNonRepeating(String s) {
    int[] freq = new int[256];
    for (char c : s.toCharArray()) freq[c]++;
    for (char c : s.toCharArray()) {
        if (freq[c] == 1) System.out.print(c + " ");
    }
}
```

Q60: Remove duplicate characters from a string.

```
public static String removeDuplicateChars(String s) {
    StringBuilder sb = new StringBuilder();
    boolean[] seen = new boolean[256];
    for (char c : s.toCharArray()) {
        if (!seen[c]) {
            seen[c] = true;
            sb.append(c);
        }
    }
    return sb.toString();
}
```

Q61: Check if two strings are Rotations of each other.

```
public static boolean isRotation(String s1, String s2) {
    return (s1.length() == s2.length()) && (s1 + s1).contains(s2);
}
```

Q62: Find maximum occurring character in a string.

```
public static char maxOccurringChar(String s) {
    int[] freq = new int[256];
    for (char c : s.toCharArray()) freq[c]++;
    int max = -1;
    char result = ' ';
    for (char c : s.toCharArray()) {
        if (freq[c] > max) {
            max = freq[c];
            result = c;
        }
    }
    return result;
}
```

Q63: Find smallest and largest word in a string.

```
public static void findSmallestAndLargestWord(String s) {
    String[] words = s.trim().split("\\s+");
    String small = words[0], large = words[0];
    for (String w : words) {
        if (w.length() < small.length()) small = w;
        if (w.length() > large.length()) large = w;
    }
    System.out.println("Smallest: " + small + ", Largest: " + large);
}
```

Q64: Calculate sum of all digits present inside an alphanumeric string.

```
public static int sumDigitsInString(String s) {
    int sum = 0;
    for (char c : s.toCharArray()) {
        if (Character.isDigit(c)) sum += c - '0';
    }
    return sum;
}
```

Q65: Check if a string is a Pangram (Contains all 26 English alphabet letters).

```
public static boolean isPangram(String s) {
    boolean[] mark = new boolean[26];
    s = s.toLowerCase();
    for (char c : s.toCharArray()) {
        if (c >= 'a' && c <= 'z') mark[c - 'a'] = true;
    }
    for (boolean b : mark) if (!b) return false;
    return true;
}
```

Q66: Find second largest element in an array.

```
public static int findSecondLargest(int[] arr) {
    int largest = Integer.MIN_VALUE, second = Integer.MIN_VALUE;
    for (int num : arr) {
        if (num > largest) {
            second = largest;
            largest = num;
        } else if (num > second && num != largest) {
            second = num;
        }
    }
    return second;
}
```

Q67: Find sum and average of all array elements.

```
public static void sumAndAverage(int[] arr) {
    long sum = 0;
    for (int num : arr) sum += num;
    double avg = (double) sum / arr.length;
    System.out.println("Sum: " + sum + ", Average: " + avg);
}
```

Q68: Count frequency of each element in an array.

```
public static void arrayFrequency(int[] arr) {
    Map<Integer, Integer> map = new HashMap<>();
    for (int num : arr) map.put(num, map.getOrDefault(num, 0) + 1);
    map.forEach((k, v) -> System.out.println(k + " -> " + v));
}
```

Q69: Sort an array in ascending order without using built-in methods (Bubble Sort).

```
public static void bubbleSort(int[] arr) {
    int n = arr.length;
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

Q70: Sort array elements in descending order.

```
public static void sortDescending(int[] arr) {
    int n = arr.length;
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] < arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

Q71: Sort first half of array in ascending order and second half in descending order.

```
public static void sortHalfAscHalfDesc(int[] arr) {
    Arrays.sort(arr);
    int n = arr.length;
    for (int i = n / 2, j = n - 1; i < j; i++, j--) {
        int temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
}
```

Q72: Find all repeating elements in an array.

```
public static void printRepeating(int[] arr) {
    Set<Integer> seen = new HashSet<>();
    Set<Integer> duplicates = new HashSet<>();
    for (int num : arr) {
        if (!seen.add(num)) duplicates.add(num);
    }
    System.out.println("Duplicates: " + duplicates);
}
```

Q73: Find all non-repeating elements in an array.

```
public static void printUnique(int[] arr) {
    Map<Integer, Integer> map = new HashMap<>();
    for (int num : arr) map.put(num, map.getOrDefault(num, 0) + 1);
    map.forEach((k, v) -> {
        if (v == 1) System.out.print(k + " ");
    });
}
```

Q74: Remove duplicate elements from an unsorted array.

```
public static int[] removeDuplicatesUnsorted(int[] arr) {
    Set<Integer> set = new LinkedHashSet<>();
    for (int num : arr) set.add(num);
    return set.stream().mapToInt(Integer::intValue).toArray();
}
```

Q75: Left rotate an array by 1 position.

```
public static void leftRotateByOne(int[] arr) {
    int temp = arr[0];
    for (int i = 0; i < arr.length - 1; i++) {
        arr[i] = arr[i + 1];
    }
    arr[arr.length - 1] = temp;
}
```

Section 4: Matrices, Arrays & Logic Building (Q76–Q100)

Q76: Find Scalar Matrix Multiplication result.

```
public static void scalarMultiply(int[][] matrix, int k) {
    for (int r = 0; r < matrix.length; r++) {
        for (int c = 0; c < matrix[0].length; c++) {
            matrix[r][c] *= k;
        }
    }
}
```

Q77: Add two matrices of identical dimensions.

```
public static int[][] addMatrices(int[][] A, int[][] B) {
    int rows = A.length, cols = A[0].length;
    int[][] C = new int[rows][cols];
    for (int r = 0; r < rows; r++) {
        for (int c = 0; c < cols; c++) {
            C[r][c] = A[r][c] + B[r][c];
        }
    }
    return C;
}
```

Q78: Multiply two matrices.

```
public static int[][] multiplyMatrices(int[][] A, int[][] B) {
    int r1 = A.length, c1 = A[0].length, c2 = B[0].length;
    int[][] C = new int[r1][c2];
    for (int i = 0; i < r1; i++) {
        for (int j = 0; j < c2; j++) {
            for (int k = 0; k < c1; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }
    return C;
}
```

Q79: Find Transpose of a Matrix.

```
public static int[][] transposeMatrix(int[][] matrix) {
    int r = matrix.length, c = matrix[0].length;
    int[][] trans = new int[c][r];
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) {
            trans[j][i] = matrix[i][j];
        }
    }
    return trans;
}
```

Q80: Check if a matrix is Symmetric (A = A^T).

```
public static boolean isSymmetricMatrix(int[][] matrix) {
    int r = matrix.length, c = matrix[0].length;
    if (r != c) return false;
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) {
            if (matrix[i][j] != matrix[j][i]) return false;
        }
    }
    return true;
}
```

Q81: Print Diagonal elements of a matrix.

```
public static void printDiagonals(int[][] matrix) {
    int n = matrix.length;
    System.out.print("Primary: ");
    for (int i = 0; i < n; i++) System.out.print(matrix[i][i] + " ");
    System.out.print("\nSecondary: ");
    for (int i = 0; i < n; i++) System.out.print(matrix[i][n - 1 - i] + " ");
}
```

Q82: Calculate Sum of Primary and Secondary Diagonals.

```
public static int sumDiagonals(int[][] matrix) {
    int n = matrix.length, sum = 0;
    for (int i = 0; i < n; i++) {
        sum += matrix[i][i];
        if (i != n - 1 - i) sum += matrix[i][n - 1 - i];
    }
    return sum;
}
```

Q83: Rotate a 2D Matrix by 90 degrees clockwise in-place.

```
public static void rotate90Clockwise(int[][] matrix) {
    int n = matrix.length;
    // Transpose
    for (int i = 0; i < n; i++) {
        for (int j = i; j < n; j++) {
            int temp = matrix[i][j];
            matrix[i][j] = matrix[j][i];
            matrix[j][i] = temp;
        }
    }
    // Reverse each row
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n / 2; j++) {
            int temp = matrix[i][j];
            matrix[i][j] = matrix[i][n - 1 - j];
            matrix[i][n - 1 - j] = temp;
        }
    }
}
```

Q84: Print Matrix in Spiral Order.

```
public static List<Integer> spiralOrder(int[][] matrix) {
    List<Integer> res = new ArrayList<>();
    int top = 0, bottom = matrix.length - 1;
    int left = 0, right = matrix[0].length - 1;
    while (top <= bottom && left <= right) {
        for (int j = left; j <= right; j++) res.add(matrix[top][j]);
        top++;
        for (int i = top; i <= bottom; i++) res.add(matrix[i][right]);
        right--;
        if (top <= bottom) {
            for (int j = right; j >= left; j--) res.add(matrix[bottom][j]);
            bottom--;
        }
        if (left <= right) {
            for (int i = bottom; i >= top; i--) res.add(matrix[i][left]);
            left++;
        }
    }
    return res;
}
```

Q85: Check if a matrix is an Identity Matrix.

```
public static boolean isIdentityMatrix(int[][] matrix) {
    int r = matrix.length, c = matrix[0].length;
    if (r != c) return false;
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) {
            if (i == j && matrix[i][j] != 1) return false;
            if (i != j && matrix[i][j] != 0) return false;
        }
    }
    return true;
}
```

Q86: Check if a matrix is Sparse (zeros > non-zero elements).

```
public static boolean isSparseMatrix(int[][] matrix) {
    int zeros = 0, total = matrix.length * matrix[0].length;
    for (int[] row : matrix) {
        for (int val : row) {
            if (val == 0) zeros++;
        }
    }
    return zeros > total / 2;
}
```

Q87: Find row with maximum number of 1s in a boolean matrix.

```
public static int rowWithMaxOnes(int[][] matrix) {
    int maxRow = -1, maxOnes = -1;
    for (int r = 0; r < matrix.length; r++) {
        int count = 0;
        for (int c = 0; c < matrix[0].length; c++) {
            if (matrix[r][c] == 1) count++;
        }
        if (count > maxOnes) {
            maxOnes = count;
            maxRow = r;
        }
    }
    return maxRow;
}
```

Q88: Count total elements present in a 2D array.

```
public static int countElements2D(int[][] matrix) {
    return matrix.length * matrix[0].length;
}
```

Q89: Find minimum and maximum element in a 2D matrix.

```
public static void findMinMax2D(int[][] matrix) {
    int min = Integer.MAX_VALUE, max = Integer.MIN_VALUE;
    for (int[] row : matrix) {
        for (int val : row) {
            min = Math.min(min, val);
            max = Math.max(max, val);
        }
    }
    System.out.println("Min: " + min + ", Max: " + max);
}
```

Q90: Calculate digital root of a non-negative integer (Keep adding digits until sum is single digit).

```
public static int addDigits(int num) {
    if (num == 0) return 0;
    return (num % 9 == 0) ? 9 : num % 9;
}
```

Q91: Determine if an integer is a Palindrome without extra string space.

```
public static boolean isPalindromeInt(int x) {
    if (x < 0 || (x % 10 == 0 && x != 0)) return false;
    int revertedNumber = 0;
    while (x > revertedNumber) {
        revertedNumber = revertedNumber * 10 + x % 10;
        x /= 10;
    }
    return x == revertedNumber || x == revertedNumber / 10;
}
```

Q92: Count frequency of digits in a given number.

```
public static void digitFrequency(int n) {
    int[] count = new int[10];
    n = Math.abs(n);
    while (n > 0) {
        count[n % 10]++;
        n /= 10;
    }
    for (int i = 0; i < 10; i++) {
        if (count[i] > 0) System.out.println(i + " -> " + count[i]);
    }
}
```

Q93: Replace all 0s with 1s in a given integer.

```
public static int replaceZerosWithOnes(int n) {
    if (n == 0) return 1;
    int result = 0, place = 1;
    while (n > 0) {
        int d = n % 10;
        if (d == 0) d = 1;
        result += d * place;
        place *= 10;
        n /= 10;
    }
    return result;
}
```

Q94: Check if a number can be expressed as sum of two prime numbers (Goldbach's Conjecture check).

```
public static void checkGoldbach(int n) {
    for (int i = 2; i <= n / 2; i++) {
        if (isPrime(i) && isPrime(n - i)) {
            System.out.println(n + " = " + i + " + " + (n - i));
        }
    }
}
```

Q95: Find total prime numbers between two given numbers L and R.

```
public static int countPrimesInRange(int L, int R) {
    int count = 0;
    for (int i = Math.max(2, L); i <= R; i++) {
        if (isPrime(i)) count++;
    }
    return count;
}
```

Q96: Generate custom Random integer numbers within a specific range [Min, Max].

```
public static int getRandomInRange(int min, int max) {
    return (int) (Math.random() * (max - min + 1)) + min;
}
```

Q97: Calculate Euclidean distance between two 2D points (x1, y1) and (x2, y2).

```
public static double euclideanDistance(double x1, double y1, double x2, double y2) {
    return Math.sqrt(Math.pow(x2 - x1, 2) + Math.pow(y2 - y1, 2));
}
```

Q98: Check if three points (x1, y1), (x2, y2), (x3, y3) lie on the same straight line (Collinear).

```
public static boolean areCollinear(int x1, int y1, int x2, int y2, int x3, int y3) {
    return (y2 - y1) * (x3 - x2) == (y3 - y2) * (x2 - x1);
}
```

Q99: Compute Area of a Triangle using Heron's Formula.

```
public static double triangleArea(double a, double b, double c) {
    double s = (a + b + c) / 2.0;
    return Math.sqrt(s * (s - a) * (s - b) * (s - c));
}
```

Q100: Check if a given integer is a Fibonacci number.